

**TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA**

**SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

SEMESTRE:

Agosto - Diciembre 2025

CARRERA:

Ingeniería Informática

MATERIA:

Patrones de diseño de software

TÍTULO ACTIVIDAD:

Examen unidad 2

UNIDAD A EVALUAR:

2

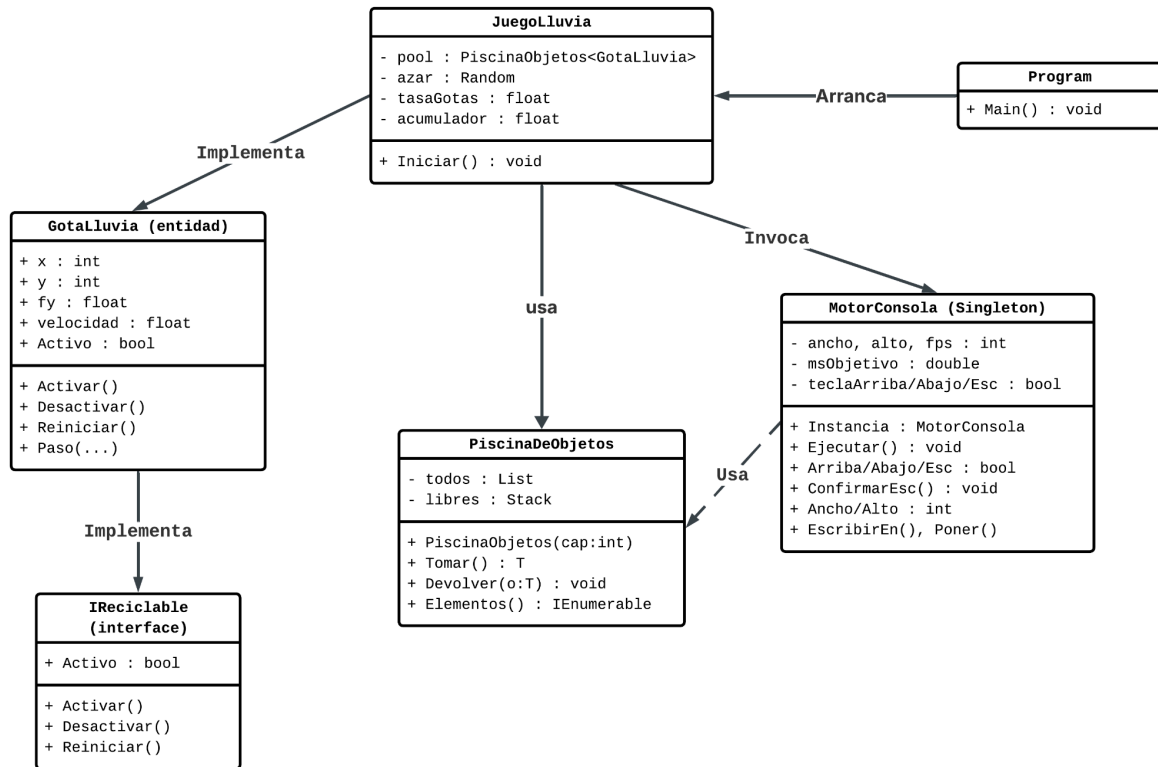
NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Herrera Sandoval Jose Miguel 21212344

NOMBRE DEL MAESTRO (A):

Maribel Guerrero Luis

Diagrama UML



Código

Interfaz IReciclable (Object Pool)

```
namespace TestExamenU2v5
{
    // Object Pool
    2 referencias
    interface IReciclable
    {
        4 referencias
        bool Activo { get; } // indica si el objeto está en uso
        2 referencias
        void Activar();
        4 referencias
        void Desactivar();
        3 referencias
        void Reiniciar(); // deja el objeto en estado inicial
    }
    3 referencias
    sealed class PiscinaObjetos<T> where T : class, IReciclable, new()
    {
        readonly List<T> todos;
        readonly Stack<T> libres;
        1 referencia
        public PiscinaObjetos(int capacidad)
        {
            todos = new List<T>(capacidad); //objetos del pool
            libres = new Stack<T>(capacidad); //disponibles para "prestar"
            for (int i = 0; i < capacidad; i++)
            {
                var o = new T();
                o.Desactivar();
                o.Reiniciar();
                todos.Add(o);
                libres.Push(o);
            }
        }
    }
}
```

```
1 referencia
public T Tomar() //saca 1 objeto de libres
{
    return libres.Count > 0 ? libres.Pop() : null;
}
1 referencia
public void Devolver(T o) //desactiva, reinicia y lo regresa a libres
{
    if (o == null)
    {
        return;
    }
    o.Desactivar();
    o.Reiniciar();
    libres.Push(o);
}
// expone la lista completa (todos) para poder dibujar los activos.
2 referencias
public IEnumerable<T> Elementos() => todos;
}
```

Clase GotaLluvia (implementa IReciclable)

```
namespace TestExamenU2v5
{
    // Particula de lluvia
    2 referencias
    sealed class GotaLluvia : IReciclable
    {
        // columnas/filas, posición vertical en float, velocidad de caída
        public int x, y; public float fy; public float velocidad; bool activo;
        4 referencias
        public bool Activo => activo;

        // implementan de IReciclable
        2 referencias
        public void Activar() { activo = true; }
        4 referencias
        public void Desactivar() { activo = false; }
        3 referencias
        public void Reiniciar()
        {
            x = y = 0;
            fy = 0f;
            velocidad = 24f;
        }

        // Actualiza la caída
        1 referencia
        public void Paso(int alto, float dt)
        {
            if (!activo)
            {
                return;
            }
            fy += velocidad * dt;
            y = (int)fy;
            // Si "y" llega al final, llama a Desactivar()
            if (y >= alto - 1)
            {
                Desactivar();
            }
        }
    }
}
```

Clase MotorConsole

```
namespace TestExamenU2v5
{
    // Motor de consola (Singleton)
    6 referencias
    sealed class MotorConsole
    {
        static readonly Lazy<MotorConsole> _inst = new Lazy<MotorConsole>(() => new MotorConsole());
        1 referencia
        public static MotorConsole Instancia => _inst.Value;

        int ancho = 80, alto = 28, fps = 30; // tamaño deseado y fps
        double msObjetivo;
        bool corriendo;
        bool teclaArriba, teclaAbajo, teclaEsc; // estados de teclas

        1 referencia
        MotorConsole()
        {
            msObjetivo = 1000.0 / fps; // ms por fotograma
        }

        1 referencia
        public void Ejecutar(Action<float> actualizar, Action dibujar, Func<bool> pedirSalir)
        {
            try
            {
                Console.CursorVisible = false; // Oculta cursor
                Console.SetWindowSize(Math.Min(ancho, Console.LargestWindowWidth), Math.Min(alto, Console.LargestWindowHeight));
                Console.SetBufferSize(ancho, alto);
            }
            catch
            {
                // Ignorar errores al cambiar el tamaño de la consola (solo si es mayor al del sistema)
            }

            // inicia cronómetro y bucle
            var cron = Stopwatch.StartNew();
            double previo = cron.Elapsed.TotalMilliseconds;
            corriendo = true;
        }
    }
}
```

```
while (corriendo)
{
    // Lectura de input
    teclaArriba = teclaAbajo = false;
    while (Console.KeyAvailable)
    {
        // asignación de teclas
        var k = Console.ReadKey(true).Key;
        if (k == ConsoleKey.UpArrow) teclaArriba = true;
        if (k == ConsoleKey.DownArrow) teclaAbajo = true;
        if (k == ConsoleKey.Escape) teclaEsc = true;
    }

    if (pedirSalir()) break; // si devuelve true, salir del bucle

    double ahora = cron.Elapsed.TotalMilliseconds;
    // calcula dt entre frames con Stopwatch
    float dt = (float)((ahora - previo) / 1000.0); previo = ahora;

    actualizar(dt); // lógica del juego

    Console.SetCursorPosition(0, 0);
    Console.Clear();
    dibujar(); // crea pantalla, gotas y suelo

    int dormir = (int)(msObjetivo - (Stopwatch.GetTimestamp() - cron.ElapsedTicks) * 1000.0 / Stopwatch.Frequency);
    if (dormir > 0)
    {
        Thread.Sleep(dormir);
    }
}

try
{
    Console.CursorVisible = true; // hace visible el cursor al salir
}
catch
{
    // Ignorar errores al cambiar la visibilidad del cursor
}
}
```

```
// Acciones
1 referencia
public bool Arriba => teclaArriba;
1 referencia
public bool Abajo => teclaAbajo;
1 referencia
public bool Esc => teclaEsc;
1 referencia
public void ConfirmarEsc()
{
    teclaEsc = false;
}

// Dimensiones
2 referencias
public int Ancho => ancho;
2 referencias
public int Alto => alto;

// escribe texto en una fila/columna (recorta si se sale).
1 referencia
public void EscribirEn(int x, int y, string s)
{
    if (y < 0 || y >= alto)
    {
        return;
    }
    if (x < 0) x = 0;
    {
        if (x + s.Length > ancho) s = s.Substring(0, Math.Max(0, ancho - x));
    }
    Console.SetCursorPosition(x, y); Console.Write(s);
}

// escribe un solo carácter en coordenadas.
2 referencias
public void Poner(int x, int y, char c)
{
    if (x < 0 || x >= ancho || y < 0 || y >= alto)
    {
        return;
    }
    Console.SetCursorPosition(x, y); Console.Write(c);
}
}
```

Clase JuegoLluvia

```
sealed class JuegoLluvia
{
    readonly PiscinaObjetos<GotaLluvia> pool = new PiscinaObjetos<GotaLluvia>(700); //capacidad
    readonly Random azar = new Random(); // para posiciones y pequeñas variaciones
    float tasaGotas = 50f; // gotas por segundo
    float acumulador = 0f; // integra la tasa continua para spawnear en entero

    1 referencia
    public void Iniciar()
    {
        var M = MotorConsola.Instancia;

        M.Ejecutar(
            actualizar: (dt) =>
            {
                if (M.Arriba) tasaGotas = Math.Min(tasaGotas + 10f, 600f);
                if (M.Abajo) tasaGotas = Math.Max(tasaGotas - 10f, 0f);

                acumulador += tasaGotas * dt;
                while (acumulador >= 1f)
                {
                    acumulador -= 1f;
                    var g = pool.Tomar();
                    if (g != null)
                    {
                        g.x = azar.Next(0, M.Ancho);
                        g.fy = 1;
                        g.y = 1;
                        g.velocidad = 16f + azar.Next(0, 6);
                        g.Activar();
                    }
                }
            }
        )
    }
}
```

```
        // Actualizar todas las gotas activas
        foreach (var g in pool.Elementos())
        {
            if (g.Activo)
            {
                g.Paso(M.Alto, dt);
                if (!g.Activo)
                {
                    pool.Devolver(g); // devuelve al pool si llegó al suelo
                }
            }
        },

        // Dibuja la pantalla
        dibujar: () =>
        {
            M.EscribirEn(0, 0, $"Lluvia por Consola | Intensidad: {(int)tasaGotas}/s (Arriba/Abajo) ESC: salir");
            foreach (var g in pool.Elementos())
            {
                if (g.Activo) M.Poner(g.x, g.y, '|');
            }
            for (int x = 0; x < M.Ancho; x++)
            {
                M.Poner(x, M.Alto - 1, '_');
            }
        },

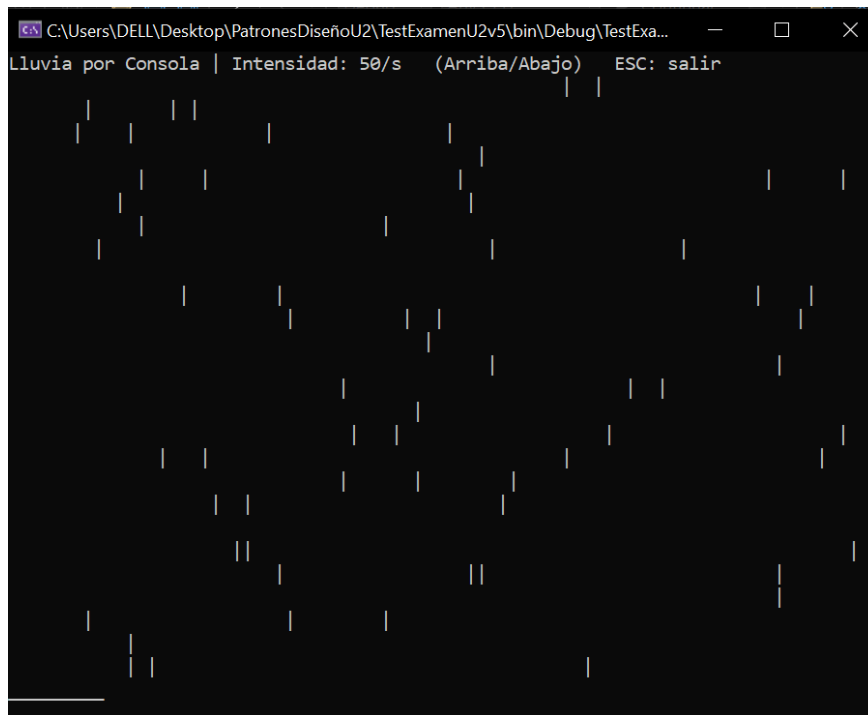
        // Solo funcionará si tecleamos ESC
        pedirSalir: () =>
        {
            if (M.Esc)
            {
                M.ConfirmarEsc();
                return true;
            }
            return false;
        }
    );
}
```

Clase Program

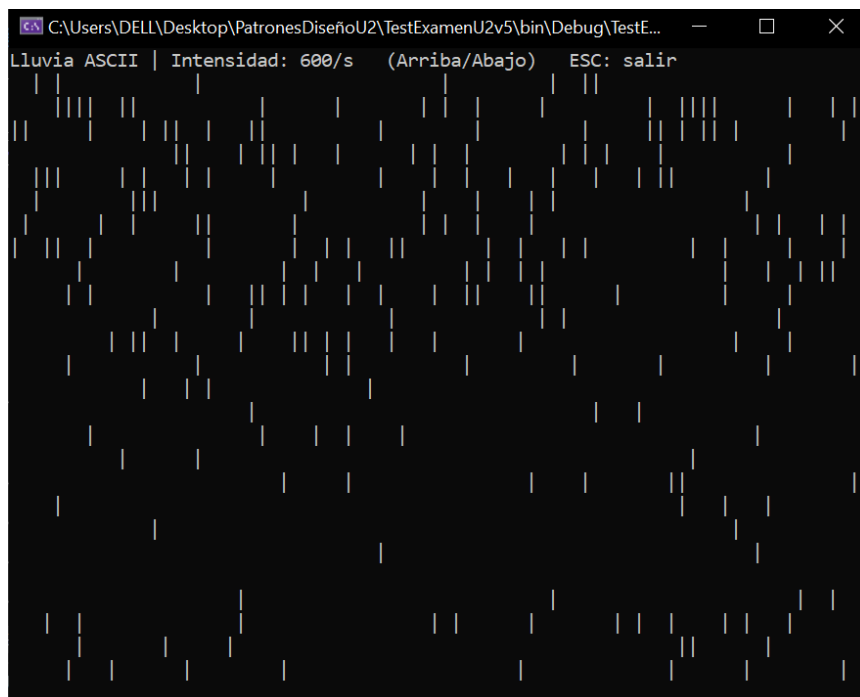
```
namespace TestExamenU2v5
{
    0 referencias
    static class Program
    {
        // Corre consola y muestra la lluvia
        0 referencias
        static void Main()
        {
            new JuegoLluvia().Iniciar();
        }
    }
}
```


Capturas de pantalla (Ejecución)

Ejecución por default (velocidad de las gotas 50/s)



Se puede modificar la velocidad de las gotas de lluvia hasta 600/s



Conclusión

Tanto el patrón Object Pool como Singleton demostraron ser muy buenos a la hora de manejar texturas (en este caso símbolos ascii) y un motor gráfico. Usar Singleton y Object Pool juntos funcionan cuando cada uno hace “lo suyo” sin estorbar al otro. Uno dirige el juego (ritmo y entradas) y el otro cuida y reutiliza los “objetos” para no desperdiciar recursos, como se observó en el documento y en la ejecución.

Esto demuestra que si se mantienen reglas simples (reutilizar en lugar de crear/romper), el programa mantendrá su fluidez y estabilidad.